

Concurrency Control

Quick Recap

- Every transaction must follow **ACID** properties .
- Concurrency control techniques are used to ensure that the *Isolation* property for concurrently executing transactions.

Why Concurrency Control management is needed?

- *Why we should have interleaving execution of transactions ?*

(if it may lead to problems such as Irrecoverable Schedule, Inconsistency and many more .)

- *Why not just Serial schedules ?*

(no complications at all.)

Why Concurrency Control management is needed?

- The performance effects the efficiency is too low which is not acceptable.
Hence a Database may provide a mechanism that ensures that the schedules are either conflict or view serializable and recoverable (also preferably cascadeless).
- Testing for a schedule for Serializability after it has executed is obviously *too late!*
So we need **Concurrency Control Protocols that ensures Serializability .**

Overview

- Different concurrency control protocols provide different advantages between *the amount of concurrency they allow and the amount of overhead that they impose.*
- Different categories of protocols:
 1. Lock Based Protocol
 - ✓ Basic 2-PL
 - ✓ Conservative 2-PL
 - ✓ Strict 2-PL
 - ✓ Rigorous 2-PL

Overview

2.Graph Based Protocol

3.Time-Stamp Ordering Protocol

4.Multiple Granularity Protocol

5.Multi-version Protocol

Lock-Based Protocols

- A lock is a mechanism to control concurrent access to a data item .
- Data items can be locked in *two modes* :
 - 1. Exclusive (X) mode** : Data item can be both read as well as written.
X-lock is requested using lock-X instruction.
 - 2. Shared (S) mode**: Data item can only be read.
S-lock is requested using lock-S instruction.

Lock-Based Protocols

- Lock requests are made to the *concurrency-control manager* by the programmer. Transaction can proceed only after request is granted.
- **Lock-compatibility matrix**

	S	X
S	true	false
X	false	false

Lock-Based Protocols

- A transaction may be granted a lock on an item if the requested lock is compatible with locks already held on the item by other transactions.
- Any number of transactions can hold shared locks on an item, But if any transaction holds an exclusive on the item no other transaction may hold any lock on the item.

Lock-Based Protocols

- If a lock cannot be granted, the requesting transaction is made to wait till all incompatible locks held by other transactions have been released. The lock is then granted

Lock-Based Protocols

- Example of a transaction performing locking:

```
T2: lock-S(A);  
    read (A);  
    unlock(A);  
    lock-S(B);  
    read (B);  
    unlock(B);  
    display(A+B)
```

- Locking as above is not sufficient to guarantee serializability — if A and B get updated in-between the read of A and B, the displayed sum would be wrong.

Lock-Based Protocols

- **Upgrade / Downgrade locks** : A transaction that holds a lock on an item **A** is allowed under certain condition to change the lock state from one state to another.
- **Upgrade**: A S(A) can be upgraded to X(A) if T_i is the only transaction holding the S-lock on element A.
- **Downgrade**: We may downgrade X(A) to S(A) when we feel that we no longer want to write on data-item A. As we were holding X-lock on A, we need not check any conditions.

Lock-Based Protocols

- But wait, just by applying locks if our problems are not solved .
- Process Synchronization under OS has one consistent problem, *starvation and Deadlock!*
- we have to apply Locks but they must follow a set of protocols to avoid such undesirable problems.
- 2-Phase Locking (2-PL) uses the concept of Locks to avoid deadlock. Applying simple locking, we may not always produce Serializable results, it may lead to Deadlock Inconsistency.

Problem With Simple Locking...

- Consider the Partial Schedule:

	T1	T2
1	lock-X(B)	
2	read(B)	
3	B:=B-50	
4	write(B)	
5		lock-S(A)
6		read(A)
7		lock-S(B)
8	lock-X(A)	
9		

Problem With Simple Locking...

- **Deadlock** – During the execution phase,
Now, T_1 holds an Exclusive lock over B,
and T_2 holds a Shared lock over A.
Consider Statement 7,
 T_2 requests for lock on B, while
in Statement 8 T_1 requests lock on A.
This imposes a **Deadlock** as none can proceed
with their execution.

Problem With Simple Locking...

- **Starvation OR Livelock** is the situation when a transaction has to wait for a indefinite period of time to acquire a lock.
- Reasons of **Starvation** – If waiting scheme for locked items is unfair.
- i.e. concurrency control manager is badly designed.

For example: A transaction may be waiting for an X-lock on an item, while a sequence of other transactions request and are granted an S-lock on the same item.
This may be avoided if the concurrency control manager is properly designed.

Two Phase Locking(2-PL)

- To guarantee serializability, we must follow some additional protocol *concerning the positioning of locking and unlocking operations* in every transaction.
- This is where the concept of Two Phase Locking(2-PL) comes into picture, 2-PL ensures serializability.

The Two-Phase Locking Protocol

- A transaction is said to follow Two Phase Locking protocol if Locking and Unlocking can be done in two phases.
- **Growing Phase:** New locks on data items may be acquired but none can be released.
- **Shrinking Phase:** Existing locks may be released but no new locks can be acquired.
- **Note** – If lock conversion is allowed, then upgrading of lock(from S(a) to X(a)) is allowed in Growing Phase and downgrading of lock (from X(a) to S(a)) must be done in shrinking phase.).

The Two-Phase Locking Protocol

	T1	T2
1	LOCK-S(A)	
2		LOCK-S(A)
3	LOCK-X(B)	
4		
5	UNLOCK(A)	
6		LOCK-X(C)
7	UNLOCK(B)	
8		UNLOCK(A)
9		UNLOCK(C)
10		

The Two-Phase Locking Protocol

- **Transaction T_1 :**
 - Growing Phase is from steps 1-3.
 - Shrinking Phase is from steps 5-7.
 - Lock Point at 3
- **Transaction T_2 :**
 - Growing Phase is from steps 2-6.
 - Shrinking Phase is from steps 8-9.
 - Lock Point at 6

The Two-Phase Locking Protocol

- What is **LOCK POINT** ?

The Point at which the growing phase ends, i.e., when transaction takes the final lock it needs to carry on its work.

- drawbacks of 2-PL.

- Cascading Rollback is possible under 2-PL.
- Deadlocks and Starvation is possible.

The Two-Phase Locking Protocol

- **The Two-Phase Locking Protocol** ensures Conflict Serializability but ***does not*** prevent Cascading Rollback and Deadlock.
- **Types of 2PL**
 - Strict 2PL,
 - Conservative 2PL and
 - Rigorous 2PL.

Strict 2-PL

- **All Exclusive(X) Locks** held by the transaction be released until *after* the Transaction Commits.
- Strict 2-PL ensures that our schedule is:
 - Recoverable
 - Cascadeless
- Still *Deadlocks are possible!*

Rigorous 2-PL

- **All Exclusive(X) and Shared(S) Locks** held by the transaction be released until *after* the Transaction Commits.
Rigorous 2-PL ensures that our schedule is
 - Recoverable
 - Cascadeless
- still *Deadlocks are possible!*
- The difference between Strict 2-PL and Rigorous 2-PL is that Rigorous is more restrictive and easy to implement.

Conservative 2-PL/ Static 2-PL

- This protocol requires the transaction to lock all the items it access before the Transaction begins execution by predeclaring its read-set and write-set.
- If any of the predeclared items needed cannot be locked, the transaction does not lock any of the items, instead it waits until all the items are available for locking.

Conservative 2-PL is **Deadlock free** and but it does not ensure Strict schedule.

- However, it is difficult to use in practice because of need to predeclare the read-set and the write-set which is not possible in many situations.
- In practice, the most popular variation of 2-PL is **Strict 2-PL**.

Timestamp based Concurrency Control

- **Timestamp** is a unique identifier created by the DBMS to identify a transaction.
- They are usually assigned in the order in which they are submitted to the system.

Timestamp Ordering Protocol

- The Timestamp Ordering Protocol is used to order the transactions based on their Timestamps. The order of transaction is nothing but the ascending order of the transaction creation.
- The priority of the older transaction is higher that's why it executes first. To determine the timestamp of the transaction, this protocol uses system time or logical counter.
- The lock-based protocol is used to manage the order between conflicting pairs among transactions at the execution time.
- But Timestamp based protocols start working as soon as a transaction is created.

Timestamp Ordering Protocol

- Let's assume there are two transactions T1 and T2.
- Suppose the transaction T1 has entered the system at 007 times and transaction T2 has entered the system at 009 times.
- T1 has the higher priority, so it executes first as it is entered the system first.
- The timestamp ordering protocol also maintains the timestamp of last 'read' and 'write' operation on a data.

Timestamp Ordering Protocol

1. Check the following condition whenever a transaction T_i issues a **Read (X)** operation:
 - If $W_TS(X) > TS(T_i)$ then the operation is rejected.
 - If $W_TS(X) \leq TS(T_i)$ then the operation is executed.
 - Timestamps of all the data items are updated.

Timestamp Ordering Protocol

2. Check the following condition whenever a transaction T_i issues a **Write(X)** operation:

- If $TS(T_i) < R_TS(X)$ then the operation is rejected.
- If $TS(T_i) < W_TS(X)$ then the operation is rejected and T_i is rolled back otherwise the operation is executed.

Timestamp Ordering Protocol

Where,

TS(Ti) denotes the timestamp of the transaction T_i .

R_TS(X) denotes the Read time-stamp of data-item X.

W_TS(X) denotes the Write time-stamp of data-item X.

Concurrency control

DEADLOCK IN DBMS

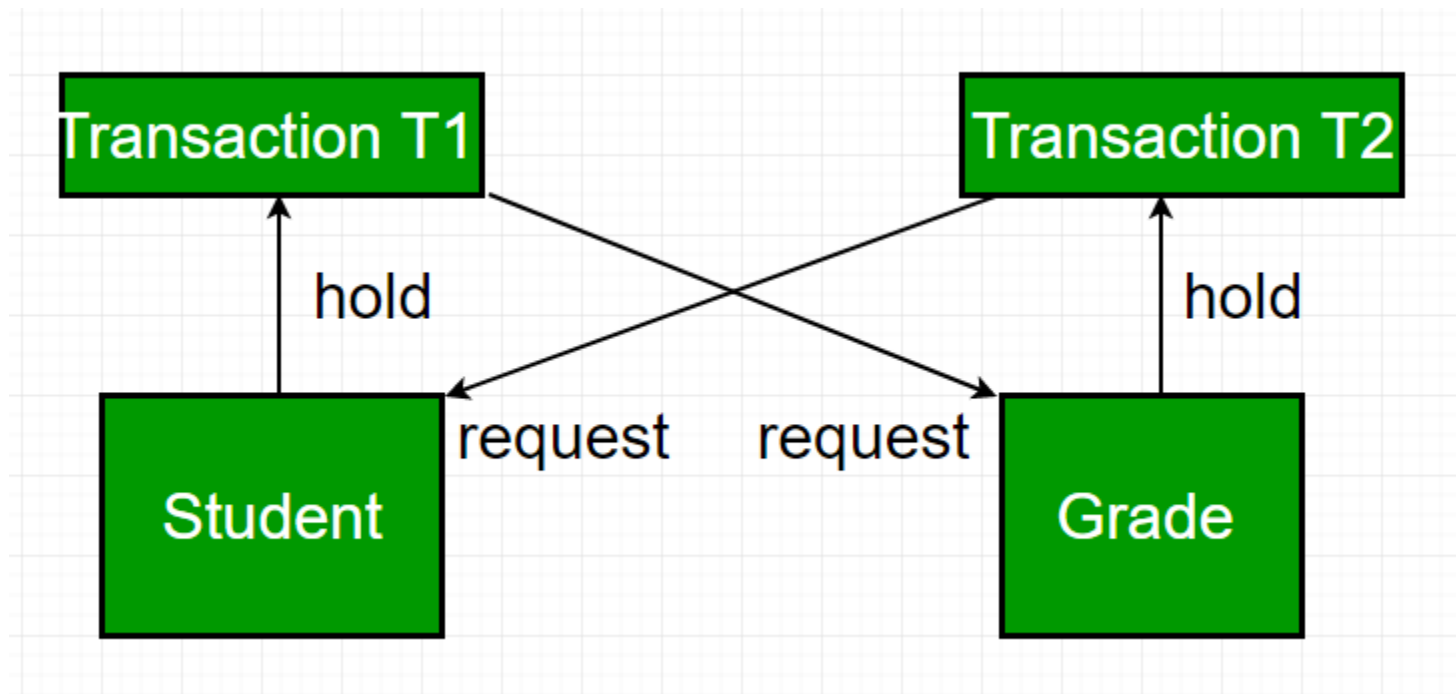
Deadlock in DBMS

- In a database, a deadlock is an unwanted situation in which two or more transactions are waiting indefinitely for one another to give up locks.
- Deadlock is said to be one of the most feared complications in DBMS as it brings the whole system to a Halt.

Deadlock in DBMS

- Suppose, Transaction T1 holds a lock on some rows in the Students table and **needs to update** some rows in the Grades table.
- Simultaneously, Transaction **T2 holds** locks on Grades table **but needs** to update the rows in the Student table **held by Transaction T1**.
- Now, the main problem arises. Transaction T1 will wait for transaction T2 to give up lock, and similarly transaction T2 will wait for transaction T1 to give up lock.

Deadlock in DBMS



Deadlock in DBMS

- As a consequence, All activity comes to a halt and remains at a standstill forever unless the DBMS detects the deadlock and aborts one of the transactions.

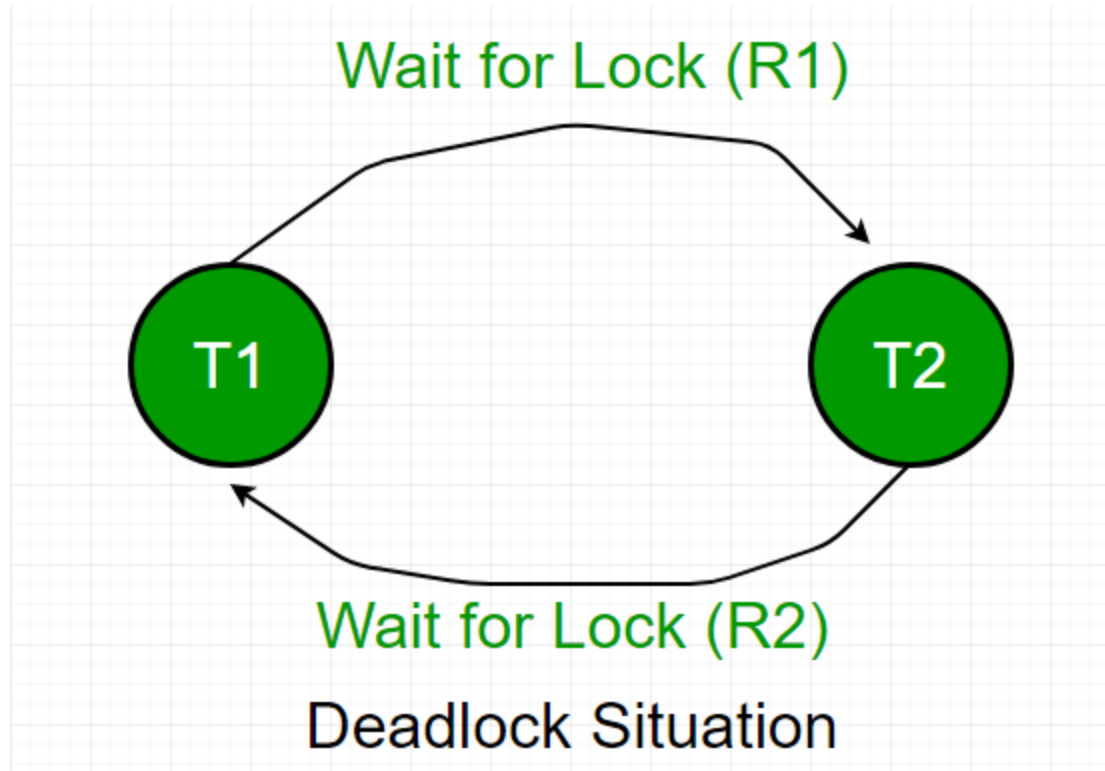
Deadlock Avoidance

- It is always better to avoid the deadlock rather than restarting or aborting the database.
- Deadlock **avoidance method is suitable for smaller database** whereas **deadlock prevention method is suitable for larger database.**
- One method of avoiding deadlock is using application consistent logic.
- In the previous example, Transaction T1 simply waits for transaction T2 to release the lock on Grades before it begins.
- When transaction T2 releases the lock, Transaction T1 can proceed freely.
- Another method for avoiding deadlock is to apply both row level locking mechanism and READ COMMITTED isolation level.
- However, It does not guarantee to remove deadlocks completely.

Deadlock Detection

- When a transaction waits indefinitely to obtain a lock, The database management system should detect whether the transaction is involved in a deadlock or not.
- **Wait-for-graph** is one of the methods for detecting the deadlock situation.
- This method is suitable for smaller database.
- In this method a graph is drawn based on the transaction and their lock on the resource.
- If the graph created has a closed loop or a cycle, then there is a deadlock.

Deadlock Detection



Deadlock prevention

- For large database, deadlock prevention method is suitable.
- A deadlock can be prevented if the resources are allocated in such a way that deadlock never occur.
- The DBMS analyzes the operations whether they can create deadlock situation or not, If they do, that transaction is never allowed to be executed.

Deadlock prevention mechanism proposes two schemes

1. Wait-Die Scheme

In this scheme, If a transaction request for a resource that is locked by other transaction, then the DBMS simply checks the timestamp of both transactions and allows the older transaction to wait until the resource is available for execution.

Deadlock prevention mechanism proposes two schemes

Suppose, there are two transactions T1 and T2 .

Let timestamp of any transaction T be $TS(T)$.

Now, If there is a lock on T2 by some other transaction and T1 is requesting for resources held by T2, then DBMS performs following actions:

Checks if $TS(T1) < TS(T2)$

if T1 is the older transaction and T2 has held some resource, then it allows T1 to wait until resource is available for execution.

Deadlock prevention mechanism proposes two schemes

That means if a younger transaction has locked some resource and older transaction is waiting for it, then older transaction is allowed wait for it till it is available.

If T1 is older transaction and has held some resource with it and if T2 is waiting for it, then T2 is killed and restarted latter with random delay but with the same timestamp.

Deadlock prevention mechanism proposes two schemes

if the older transaction has held some resource and younger transaction waits for the resource, then younger transaction is killed and restarted with very minute delay with same timestamp.

This scheme allows the older transaction to wait but kills the younger one.

Deadlock prevention mechanism proposes two schemes

2.Wound Wait Scheme

In this scheme, if an older transaction requests for a resource held by younger transaction, then older transaction forces younger transaction to kill the transaction and release the resource.

- The younger transaction is restarted with minute delay but with same timestamp.
- If the younger transaction is requesting a resource which is held by older one, then younger transaction is asked to wait till older releases it.